Khởi động dự án Order Flow

https://gemini.google.com/app/736dff8dc878c5be

User prompt: i need to review all other best opensource about this type website and can give the ideas to update it more for our purpose and then give me the flow of this website full flow, and have any issues, and the solutions for them

Response: To build a high-performance Order Flow + AI Radar platform without reinventing the wheel, we should extract the best parts of existing open-source repositories, modify them to fit your requirements, and connect them into a cohesive pipeline.

Part 1: Review of the Best Open-Source Repositories & How to Upgrade Them

No single open-source project has **both** Order Flow charting and an **AI Opportunity Radar**. However, the four industry-standard open-source projects below solve 80% of the data and charting work.

Repository	Tech Stack	What It Does Best	Limitations for Your Goal	How We Upgrade It for Your Project
1. `Tucsky/aggr` & `aggr-server` (Aggr.trade)	Vue.js, TypeScript, Web Workers, Node.js	The gold standard for real-time crypto order flow. Uses **Web Workers** to aggregate live trades, CVD, and liquidations across 15+ exchanges without lagging the browser.	Charts are basic custom panes; no TPO/Footprint ladders; no AI integration.	**Extract the Web Worker & `aggr-server` logic** to collect real-time trades (for tick-accurate CVD/Delta) alongside your CoinGlass API feed.
2. `awahab07/lightweight-orderflow-charts`	TypeScript, React, TradingView Lightweight Charts v5	Bolts directly onto TradingView Lightweight Charts. Renders **Footprints, TPO / Volume Profiles, POC, Stacked Imbalances, Absorption, and CVD subcharts** natively.	Frontend rendering library only—requires you to supply normalized JSON (`bars-1m.json` / `orderflow-1m.json`).	**Use as your primary Frontend Charting Core.** Add a custom overlay layer to draw AI Entry, Stop Loss, and Take Profit boxes directly onto the chart.
3. `flowsurface-rs/flowsurface` & `flowsurface-server`	Rust, DuckDB, Apache Arrow	Institutional-grade TPO (Daily/Weekly/Monthly with Value Areas, Single Prints) and historical DOM Heatmaps. Includes a collector that writes trades to **DuckDB**.	Native desktop app (Rust Iced GUI), not a web app.	**Port its TPO calculation math & DuckDB/TimescaleDB storage schema** to your backend to compute Value Area High (VAH), Low (VAL), and POC rapidly.
4. `OrderFlow-Analysis-Pro`	Python	Backend rule engine that detects algorithmic order flow patterns: **Absorption, Liquidity Sweeps, and Delta Divergence**.	Designed for single-exchange scripts; no web dashboard or LLM connection.	**Use as the Pre-Processor for DeepSeek.** Run this Python logic every 5–15 minutes to convert raw numbers into structured JSON prompts for DeepSeek V4.1 Flash.

Key Feature Upgrades to Make Your Platform Stand Out

1. **Visualizing AI Setups on the Chart:** Instead of showing the AI Opportunity Radar as a boring text box, automatically plot **Green/Red Risk-to-Reward boxes** (Entry, TP, SL, Invalidation line) right on the TradingView Lightweight Chart when the user clicks an AI signal.
2. **Event-Driven AI Triggers (Cost Optimization):** Instead of calling DeepSeek every 5 minutes on a blind timer, have the Python pre-processor watch for anomalies first (e.g., *Liquidation spike > $5M* OR *CVD Divergence at TPO POC*). Only call the DeepSeek V4.1 Flash API when an anomaly actually happens—cutting AI token costs by ~70%.
3. **Live Trade Setup Tracker (Auto-Invalidation):** Once DeepSeek outputs a setup, your backend monitors the live price. If price hits the `invalidation_condition` before reaching `entry_zone`, the system automatically marks the card as **"Invalidated"** in real time.

Part 2: Complete End-to-End System Flow

The platform operates across four distinct layers moving data from raw market feeds to the user's screen:

1 **Layer 1: Data Ingestion & Aggregation**

CoinGlass Standard API + Exchange WebSockets

- **CoinGlass Poller (REST/WS):** Pulls macro derivatives data every 10–60 seconds: Open Interest (OI), Funding Rates, Liquidation Heatmaps/Levels, and Long/Short Ratios.
- **Exchange Tick Collector (Adapted from `aggr-server`):** Connects to free Binance/Bybit WebSockets to aggregate sub-second market buys vs. market sells. This builds accurate 1-minute Footprint clusters, Volume Delta, and Cumulative Volume Delta (CVD) without burning your CoinGlass rate limits.

2 **Layer 2: Storage & Quantitative Pre-Processing**

PostgreSQL (TimescaleDB) + Redis + Python Quant Engine

- **Hot Cache (Redis):** Stores the latest state of the market (current price, live 24h TPO profile, active AI setups) for instant sub-50ms delivery to website visitors.
- **Time-Series DB (TimescaleDB):** Stores historical 1m/5m/1h/4h OHLCV, Delta, OI, and TPO arrays.
- **Quant Feature Extractor (Python):** Calculates concrete structural metrics:
  - **TPO Metrics:** Point of Control (POC), Value Area High (VAH), Value Area Low (VAL).
  - **Order Flow Signals:** CVD slope vs. Price slope (detects Absorption & Divergence), OI % change over 1h/4h, and nearest Liquidation magnet clusters.

3 **Layer 3: AI Opportunity Radar Engine**

DeepSeek V4.1 Flash + Deterministic Validator

- **Step 3A (State Vector Builder):** Packages the Quant Engine's output into a compact ~500-token JSON State Vector (separating Intraday 15m data and High Time Frame 4H/1D data).
- **Step 3B (DeepSeek V4.1 Flash Inference):** Sends the State Vector with a strict System Prompt forcing Chain-of-Thought order flow reasoning and structured JSON output (`trigger`, `invalidation`, `entry`, `tp`, `sl`, `rr_ratio`).
- **Step 3C (Safety Guardrail & State Tracker):** A backend script checks the math: verifies `R:R >= 2.0`, ensures `SL` is placed logically beyond the TPO/liquidation barrier, and broadcasts valid setups to connected users.

4 **Layer 4: Frontend Client Delivery**

Next.js/React + Web Workers + Lightweight OrderFlow Charts

- **Web Worker Stream:** Receives compressed deltas from your backend WebSocket, buffers them in 250ms batches, and hands normalized data to the UI thread.
- **Split-Screen Workspace:**
  - **Left (70% width):** TradingView Lightweight Charts + `lightweight-orderflow-charts` plugin showing K-Line, TPO Profile, CVD, OI, and Funding sub-panes.
  - **Right (30% width):** Live **AI Opportunity Radar** feed filterable by *Intraday (Scalp/Day)* vs. *HTF (Swing)*, with one-click chart overlay plotting.

Part 3: Full System Issues, Bottlenecks & Exact Engineering Solutions

Beyond the frontend rendering challenges covered earlier, here are the **system-wide issues** across data, backend, and AI—along with their architectural solutions:

System Issue	Why It Happens	Concrete Engineering Solution
1. CoinGlass Rate Limit Exhaustion	If 500 users open your site and each user's browser triggers a backend call to CoinGlass, your $299/mo Standard API quota will be blocked in 2 minutes.	**Single-Producer / Multi-Consumer Pattern:** Users **never** trigger CoinGlass calls. One background worker fetches CoinGlass data on a fixed schedule, saves it to **Redis**, and your backend serves 1,000+ users directly from Redis.
2. TPO & Footprint Calculation Lag	Calculating TPO letters and price-level volume footprints across 30 days of tick data on the fly takes several seconds of heavy CPU work.	**Materialized Buckets:** Pre-aggregate footprints into 1-minute and 15-minute JSON blocks inside TimescaleDB as candles close. When a user loads a chart, serve pre-computed blocks instead of recalculating from raw ticks.
3. AI "Hallucinated" Price Levels	LLMs sometimes invent support/resistance numbers that don't match the current price (e.g., suggesting an Entry at $62,000 when BTC is at $75,000).	**Anchor-Constrained Prompting:** Do not ask DeepSeek to pick arbitrary prices. Pass an explicit menu of calculated structural levels in the prompt (e.g., `VAL: 74200, POC: 74850, Liq_Cluster_High: 76100`) and instruct DeepSeek to anchor Entries, Stops, and Targets **strictly** around those verified levels.
4. Conflicting Intraday vs. HTF Signals	The 15m AI scan might scream "SHORT", while the 4H AI scan says "STRONG LONG", confusing the user.	**Hierarchical Context Injection:** Always inject the latest **HTF (4H/1D) Market Bias** into the **Intraday (15m) prompt**. Instruct DeepSeek: *"If the Intraday setup is counter-trend to the HTF bias, require a higher confidence threshold and tighten Take Profit targets to the nearest POC."*
5. Outdated / Zombie AI Signals	An AI trade setup generated 2 hours ago is useless if price already pumped past Take Profit or crashed through Stop Loss.	**State-Machine Lifecycle:** Every AI setup in your database has a status: `PENDING_TRIGGER` → `ACTIVE` → `HIT_TP` / `HIT_SL` / `INVALIDATED`. A lightweight price listener updates these statuses in real time so users see live win/loss tracking.

Interactive System Architecture & Data Flow Explorer

Click through the 6 architectural nodes below or step through the pipeline to inspect the exact inputs, processing logic, latency budgets, and failure safeguards at each stage of your website.

User prompt: the real time here in this system is?

Response: Depends on whether you mean the **system clock** right now, or how **"real-time" latency** works inside the Order Flow architecture we just designed:

1. Current System Clock

- **Current System Time:** Wednesday, October 21, 2026 at **8:00 PM** (UTC+7 / Indochina Time).
- Equivalent UTC Time: Wednesday, October 21, 2026 at **1:00 PM UTC**.

*(Note: When building your trading platform, always store and process all timestamps in **UTC Unix milliseconds** in your backend and database, and only convert to local time like UTC+7 on the frontend browser so timeframes like the Daily/4H candle close stay synced globally.)*

2. If You Mean "Real-Time" Latency Inside Your Order Flow Website

If you are asking what "real-time" means for the data pipeline in our diagram above, different modules run at different speeds so your server and browser don't crash:

- **Exchange WebSocket Stream (Trades, Price, CVD): ~50ms to 250ms** (True Real-Time). Your backend receives trades instantly via WebSocket, and your frontend Web Worker batches and renders them 4 times per second (every 250ms) so it feels instantaneous to the human eye without freezing the browser.
- **CoinGlass Standard API (OI, Funding Rate, Liquidations): Every 5 to 30 seconds** (Near Real-Time). Derivatives metrics don't need millisecond updates, and polling every 10–30 seconds into Redis keeps you safely inside your $299/month rate limit.
- **AI Opportunity Radar (DeepSeek V4.1 Flash): 1.5 to 3 seconds per analysis** (Event-Driven or Scheduled every 5–15 minutes). The AI does not run on every tick—it runs when a candle closes or when your Python engine detects an order flow anomaly, while a fast background script monitors the live price in real time (under 50ms) to check if an AI setup hits its Trigger, Take Profit, or Stop Loss.

User prompt: you have research all open source that can be use for our project and learning from all special things from it ? we not need to build again while we have them

Response: Here's the plan I've put together. Let me know if you need any changes before I start researching.

User prompt: Start research

Response: I've completed your research. Feel free to ask me follow-up questions or request changes.

User prompt: ok bay gio kien truc roi techstack nhu nao thiet ke kien truc ra sao

Response: Dựa trên các nghiên cứu chuyên sâu về các dự án mã nguồn mở hàng đầu (`lightweight-orderflow-charts`, `aggr-server`, `cryexc-backend`, `LLM_trader`), đây là bản thiết kế kiến trúc toàn diện và ngăn xếp công nghệ (tech stack) tối ưu cho nền tảng Order Flow & AI Opportunity Radar của bạn.

Mục tiêu cốt lõi: **Không viết lại những thứ mã nguồn mở đã làm tốt**, xử lý luồng dữ liệu thời gian thực cực mượt mà không làm đơ trình duyệt, và ép AI (DeepSeek V4.1 Flash) đưa ra các kế hoạch giao dịch chuẩn xác về mặt toán học.

1. Ngăn Xếp Công Nghệ (Tech Stack) Đề Xuất

Tầng (Layer)	Công Nghệ Lựa Chọn	Lý Do Kỹ Thuật & Kế Thừa Mã Nguồn Mở
Frontend UI	**Next.js (React) + TypeScript + Tailwind CSS**	Dễ quản lý trạng thái, tối ưu SEO cho trang đích và tạo dashboard chuyên nghiệp.
Biểu đồ Dòng Lệnh	**TradingView Lightweight Charts v5** + **`lightweight-orderflow-charts`**	Thư viện mở mở rộng trực tiếp trên Lightweight Charts: vẽ Footprint, TPO/Market Profile, CVD, Delta Summary bằng Canvas 2D mà không tốn công tự code từ đầu.
Frontend Concurrency	**HTML5 Web Workers** (Kế thừa từ `aggr`)	Đưa toàn bộ việc nhận và giải nén WebSocket ra một luồng ngầm (background thread), chỉ đẩy dữ liệu đã gộp sang UI để tránh giật lag trình duyệt.
Backend Thu Thập & API	**FastAPI (Python) + Node.js Worker**	Python xử lý cực tốt tính toán khoa học, AI và Pydantic; Node.js xử lý kết nối WebSocket tập trung với độ trễ thấp.
Cơ Sở Dữ Liệu Thời Gian Thực	**Redis (Valkey)**	Đóng vai trò Hot-Cache lưu trạng thái giá hiện tại, 24h TPO và làm Pub/Sub để phát tán dữ liệu tức thời tới các máy khách.

Tầng (Layer)	Công Nghệ Lựa Chọn	Lý Do Kỹ Thuật & Kế Thừa Mã Nguồn Mở
Cơ Sở Dữ Liệu Lịch Sử & Tính Toán	**DuckDB + PostgreSQL (TimescaleDB)**	DuckDB (học từ `cryexc-backend`) giữ dữ liệu tick trong RAM để tính toán Footprint/CVD siêu tốc; TimescaleDB lưu trữ nến lịch sử và lịch sử tín hiệu của AI.
Động Cơ AI (Radar Engine)	**DeepSeek V4.1 Flash API + Pydantic Schema**	Giá thành API cực rẻ, tốc độ suy luận nhanh; sử dụng Pydantic để khóa chặt định dạng JSON đầu ra (Trigger, Entry, SL, TP, R:R).

2. Thiết Kế Kiến Trúc Hệ Thống (4 Tầng)

Hệ thống được tổ chức theo kiến trúc phân tán hướng sự kiện (Event-Driven Architecture) gồm 4 phân tầng độc lập:

Tầng 1: Thu Thập & Chuẩn Hóa Dữ Liệu (Data Ingestion Layer)

- **CoinGlass Standard API Worker:** Chạy ngầm định kỳ gọi CoinGlass API lấy dữ liệu phái sinh vĩ mô: **Open Interest (OI)**, **Funding Rate**, **Liquidation Heatmap/Levels**, và **Long/Short Ratio**.
    - *Nguyên tắc:* Toàn bộ dữ liệu CoinGlass được lưu thẳng vào **Redis**. Trình duyệt của người dùng tuyệt đối không gọi trực tiếp vào CoinGlass để tránh bị khóa giới hạn tần suất (Rate Limit).
- **Exchange WebSocket Collector (Học từ `aggr-server`):** Kết nối WebSocket trực tiếp đến Binance/Bybit để lấy luồng khớp lệnh thực tế (`aggTrade`). Bộ thu thập này gom các lệnh mua/bán chủ động lại thành các block 1 giây / 1 phút để tính toán **Volume Delta** và **CVD** mà không tốn thêm chi phí API.

Tầng 2: Động Cơ Tiền Xử Lý Định Lượng (Quant Pre-Processing Engine)

Đây là "bộ lọc" trung gian giữa dữ liệu thô và AI.

- **Bộ nhớ DuckDB trong RAM:** Gom các lệnh thành cấu trúc `OrderFlowBar` (mỗi mức giá có bao nhiêu khối lượng mua/bán).
- **Bộ tính toán TPO:** Tính ra các mức hỗ trợ/kháng cự vi mô quan trọng: **POC (Point of Control)**, **VAH (Value Area High)**, và **VAL (Value Area Low)**.
- **Bộ phát hiện phân kỳ CVD (CVD Divergence Detector):** Thuật toán tự động so sánh xu hướng giá và xu hướng CVD. Khi giá tạo đáy thấp hơn nhưng CVD tạo đáy cao hơn (người bán bị hấp thụ), một cờ cảnh báo (Flag) được bật lên.
- **Tạo State Vector:** Nén toàn bộ dữ liệu thị trường phức tạp thành một đoạn mã JSON ngắn gọn (~400 token) chứa đầy đủ bối cảnh thị trường.

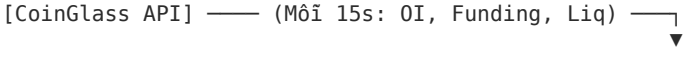
Tầng 3: AI Opportunity Radar & Bộ Kiểm Định An Toàn

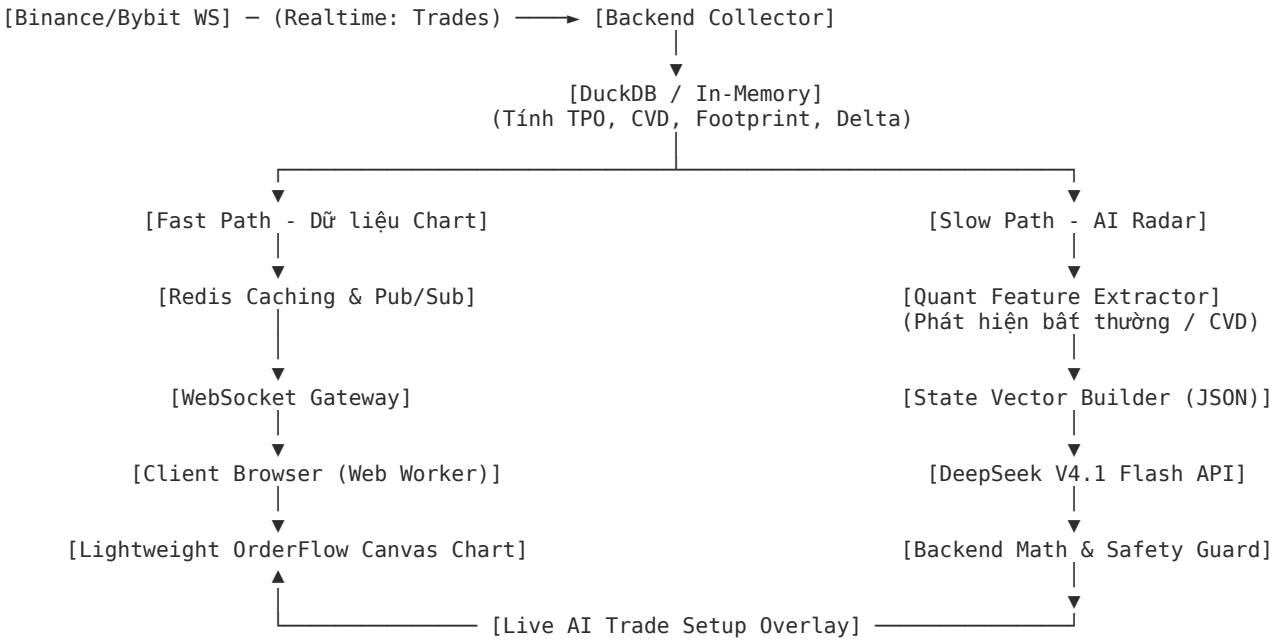
- **Cơ chế Kích hoạt (Event-Driven Trigger):** Thay vì gọi DeepSeek liên tục mỗi 3 phút gây tốn chi phí, hệ thống chỉ gọi DeepSeek khi:
    - Cây nến khung thời gian 15m (Intraday) hoặc 4H (HTF) đóng cửa.
    - Hoặc khi Động cơ Tiền Xử Lý phát hiện một bất thường dòng lệnh (ví dụ: quét thanh lý lớn hoặc xuất hiện phân kỳ CVD tại vùng POC).
- **Gửi Prompt tới DeepSeek V4.1 Flash:** Gửi JSON State Vector kèm chỉ thị nghiêm ngặt ép buộc DeepSeek suy luận theo phương pháp Order Flow.
- **Bộ Kiểm Định Xác Thực Toán Học (Deterministic Guardrail):** Khi DeepSeek trả về thiết lập giao dịch (Trade Setup), backend sẽ kiểm tra:
    - Tỷ lệ $R:R$ có đạt từ $1:2.0$ trở lên không?
    - Điểm Stop Loss (SL) có được đặt hợp lý ngoài vùng giá trị TPO không?
    - Nếu đạt chuẩn, lưu vào database và chuyển sang trạng thái `PENDING_TRIGGER`.

Tầng 4: Phân Phối & Giao Diện Người Dùng (Frontend Client Layer)

- **WebSocket Hub (Node.js/FastAPI):** Sử dụng Redis Pub/Sub để phát tán dữ liệu nến, Order Flow và tín hiệu AI mới nhất đến hàng ngàn người dùng qua kết nối WebSocket.
- **Bố cục Giao diện (Workspace):**
    - **Khu vực trung tâm (70%):** Biểu đồ TradingView Lightweight Charts hiển thị nến, biểu đồ con CVD, và các cột khối lượng Footprint từ `lightweight-orderflow-charts`.
    - **Khu vực Radar bên phải (30%):** Danh sách các cơ hội do AI quét được. Khi người dùng bấm vào một cơ hội, hệ thống tự động vẽ các khối hộp Entry, Take Profit, Stop Loss màu xanh/đỏ trực tiếp lên biểu đồ.

3. Bản Thiết Kế Dòng Chảy Dữ Liệu Chi Tiết (End-to-End Data Flow)





4. Hợp Đồng Dữ Liệu (JSON Data Contracts) Chuẩn Hóa

A. Dữ liệu nén gửi cho DeepSeek (State Vector Prompt)

```
{
  "symbol": "BTCUSDT",
  "timeframe": "15m",
  "market_bias_htf": "BULLISH_CONTINUATION",
  "current_price": 64250.0,
  "tpo_levels": {
    "vah": 64800.0,
    "poc": 64100.0,
    "val": 63650.0
  },
  "order_flow_indicators": {
    "cvd_divergence": "BULLISH_ABSORPTION_AT_POC",
    "open_interest_delta_1h": "+8.4%",
    "funding_rate": 0.0085,
    "liquidation_magnets": {
      "short_liquidation_pool": 65400.0,
      "long_liquidation_pool": 63200.0
    }
  }
}
```

B. Định dạng JSON DeepSeek bắt buộc phải trả về

```
{
  "status": "OPPORTUNITY_FOUND",
  "direction": "LONG",
  "confidence": 0.88,
  "rationale": "Gia kiem tra lai vung POC tai 64100 va xuat hien su hap thu luc ban manh voi phan ky CVD tang.
  "trade_setup": {
    "trigger_condition": "Nen 5m dong cua xac nhan tren 64300 kem Volume Delta duong",
    "invalidation_condition": "Gia pha thung va duy tri duoi muc VAL 63650 truoc khi kich hoat lenh",
    "entry_range": [64150.0, 64300.0],
    "stop_loss": 63550.0,
    "take_profit_targets": [64800.0, 65400.0],
    "risk_reward_ratio": 2.2
  }
}
```

5. Cấu Hình Hạ Tầng Đề Xuất (VPS/Server)

Để hệ thống vận hành trơn tru ở giai đoạn phát triển và thử nghiệm ban đầu (MVP), bạn nên chuẩn bị:

- **VPS Linux (Ubuntu 22.04 LTS hoặc 24.04 LTS)**

- **Cấu hình tối thiểu:**
  - **CPU:** 4 vCPU (để xử lý mượt các phép tính toán ma trận TPO và các tác vụ bất đồng bộ).
  - **RAM:** 8 GB (DuckDB và Redis hoạt động trên RAM để lưu trữ dữ liệu dòng lệnh và cache).
  - **Ổ cứng (NVMe SSD):** 80 GB - 100 GB (đảm bảo tốc độ đọc ghi I/O cho TimescaleDB).
- **Nhà cung cấp đề xuất:** Hetzner Cloud (gói CPX31 khoảng 15-20 EUR/tháng) hoặc DigitalOcean/Vultr với cấu hình tương đương. Chi phí này rất tiết kiệm và hiệu năng vượt trội cho hệ thống tính toán dữ liệu.

User prompt: phan backend khong su dung go duoc a

Response: Hoàn toàn được, và thực tế việc sử dụng **Go (Golang)** cho Backend không những được mà còn là **tiêu chuẩn vàng** cho các hệ thống tài chính tần suất cao (HFT) và Order Flow.

Lý do tôi đề xuất Python + Node.js ban đầu là vì nó dễ tiếp cận nhất cho việc tích hợp AI. Tuy nhiên, nếu bạn có khả năng triển khai Go, hệ thống của bạn sẽ được nâng cấp lên mức độ chịu tải (performance) của các tổ chức tài chính chuyên nghiệp.

Trong kiến trúc hệ thống giao dịch hiện đại, chúng ta không dùng một ngôn ngữ cho mọi thứ, mà áp dụng **Kiến trúc Vi dịch vụ (Microservices)**. Dưới đây là cách phân chia hoàn hảo giữa Go và Python cho dự án của bạn:

1. Nơi Go (Golang) Tỏa Sáng: Lớp Dữ liệu & Mạng (Tầng 1 & Tầng 4)

Go sinh ra để xử lý mạng và đồng thời (concurrency). Bạn sẽ dùng Go để thay thế hoàn toàn Node.js trong việc xử lý luồng dữ liệu thời gian thực.

- **Bộ thu thập dữ liệu (Tick Collector):** Go kết nối WebSockets tới Binance/Bybit. Nhờ cơ chế `Goroutines` và `Channels`, Go có thể xử lý hàng chục ngàn lệnh (trades) mỗi giây mà không bị đơ hoặc gặp hiện tượng giật lag do dọn rác bộ nhớ (Garbage Collection) như Node.js hay Python. *(Dự án mã nguồn mở `edgedepth-gateway` được viết hoàn toàn bằng Go là một minh chứng xuất sắc cho việc này).*
- **WebSocket Gateway cho Client:** Khi hàng ngàn người dùng truy cập website của bạn để xem biểu đồ, máy chủ Go sẽ đóng vai trò lấy dữ liệu từ Redis và "bơm" trực tiếp đến trình duyệt của người dùng với độ trễ tính bằng micro-giây.

2. Nơi Python Bắt Buộc Phải Giữ Lại: Lớp Trí tuệ AI & Toán học (Tầng 2 & Tầng 3)

Mặc dù Go cực kỳ nhanh, nhưng hệ sinh thái thư viện AI và Data Science của Go rất nghèo nàn. Bạn không nên dùng Go để tính toán lượng tử hay gọi DeepSeek.

- **Tính toán Quant (TPO, Phân kỳ CVD):** Python có `NumPy`, `Pandas` và `DuckDB` giúp tính toán các ma trận khối lượng và Vùng Giá Trị (Value Area) chỉ trong vài dòng code. Nếu tự viết các ma trận này bằng Go, sẽ rất dài và dễ sinh lỗi logic.
- **Kết nối DeepSeek & RAG:** Python sở hữu `Pydantic` (để ép định dạng JSON đầu ra của AI) và các thư viện Vector Database (ChromaDB) để làm bộ nhớ cho AI. Lớp dịch vụ Python sẽ chạy độc lập, lắng nghe tín hiệu từ Redis, gõ cửa API DeepSeek V4.1, và ném kết quả giao dịch (Entry, SL, TP) ngược lại vào database.

Kiến Trúc Mới: Tối ưu với Go + Python

Sự kết hợp này mang lại tốc độ tuyệt đối của C/C++ (từ Go) và sức mạnh AI linh hoạt (từ Python):

1. **Dữ liệu vào (Go Service):** Lắng nghe luồng lệnh từ Binance → Đóng gói gọn nhẹ → Ghi tốc độ cao vào Redis & TimescaleDB.
2. **Động cơ AI (Python Service):** Chạy ngầm, đọc dữ liệu nến 15m/1H từ cơ sở dữ liệu → Xử lý toán học định lượng → Gửi prompt cho DeepSeek Flash → Lưu Kế hoạch giao dịch vào Redis.
3. **Dữ liệu ra (Go Gateway):** Lấy luồng giá hiện tại và các Kế hoạch giao dịch từ Redis → Bắn qua WebSocket tới trình duyệt Next.js của người dùng.

User prompt: vay cho t hoan chinh kien truc va techstack chi tiet di , vi du frontend co dung lib gi khong t muon dung tanstack thi duoc khong

Response: Sử dụng **TanStack** không chỉ "được" mà còn là **sự lựa chọn xuất sắc nhất** hiện nay cho một nền tảng dữ liệu tài chính nặng. TanStack Query (trước đây là React Query) kết hợp với WebSockets sẽ giúp bạn giải quyết triệt để bài toán quản lý bộ nhớ đệm (caching) và cập nhật dữ liệu thời gian thực mà không làm tràn RAM trình duyệt.

Dưới đây là **Bản thiết kế Kiến trúc & Tech Stack chi tiết 100%** cho toàn bộ hệ thống, kết hợp Frontend (React + TanStack), Backend (Go + Python) và Database theo đúng chuẩn công nghiệp HFT (High-Frequency Trading).

1. FRONTEND: Lớp Giao diện & Hiển thị Trực quan (Client-Side)

Nhiệm vụ: Vẽ biểu đồ siêu tốc, quản lý trạng thái dữ liệu mượt mà, không giật lag.

- **Core Framework: Next.js (App Router)** hoặc **Vite + React**. (Nên chọn Vite nếu trang web thuần là một Dashboard/Web-app, chọn Next.js nếu cần SEO cho trang chủ).

- **Quản lý Dữ liệu (Data Fetching & State):**
    - **TanStack Query (React Query):** Quản lý toàn bộ API calls (lấy lịch sử nến, danh sách tín hiệu AI). Khi có dữ liệu mới từ WebSocket bắn về, ta dùng `queryClient.setQueryData()` để cập nhật trực tiếp vào bộ nhớ đệm của TanStack mà không cần gọi lại API.
    - **Zustand:** Dùng để quản lý các trạng thái UI cục bộ (ví dụ: đang chọn khung thời gian nào, đang bật/tắt chế độ Radar nào) vì nó nhẹ hơn Redux rất nhiều.
- **Bảng dữ liệu AI Radar: TanStack Table**. Xử lý việc sắp xếp (sort), lọc (filter) hàng ngàn tín hiệu AI (theo R:R, theo tỷ lệ win) với hiệu năng cực cao.
- **Thư viện Biểu đồ (Charting):**
    - **`lightweight-charts`** (của TradingView) làm lõi vẽ K-line.
    - **`lightweight-orderflow-charts`** (Mã nguồn mở): Plugin gắn thêm vào để vẽ Footprint, TPO (Market Profile), và CVD.
    - **`kline-orderbook-chart`** (Mã nguồn mở): Dùng riêng nếu bạn muốn vẽ Liquidation Heatmap (Bản đồ nhiệt thanh lý) đè lên biểu đồ.
- **Xử lý Đa luồng (Concurrency): HTML5 Web Workers**. Tách việc nhận/giải nén cục dữ liệu WebSocket ra khỏi luồng chính (Main Thread) của React để UI không bị đơ.
- **UI/Styling: Tailwind CSS** + **Shadcn UI** (cho các component như Dropdown, Modal, Toast mượt và chuyên nghiệp).

2. BACKEND GO (Golang): Lớp Gateway & Truyền tải Thời gian thực

Nhiệm vụ: Đứng mũi chịu sào, hứng hàng chục ngàn thao tác khớp lệnh mỗi giây từ sàn, gom lại và đẩy siêu tốc cho người dùng.

- **Framework: Fiber** hoặc **Gin** (Fiber cực nhanh, cú pháp giống Express.js).
- **Xử lý WebSockets: `gorilla/websocket`** (thư viện WS chuẩn mực nhất của Go).
- **Logic Dòng lệnh:** Sử dụng kiến trúc của dự án mã nguồn mở **`edgedepth-gateway`**.
    - Lắng nghe trực tiếp luồng `@aggTrade` và `@depth` từ Binance/Bybit.
    - Gom nhóm (Batching) các lệnh mua/bán chủ động trong mỗi 100ms - 250ms.
    - Đóng gói thành định dạng nhị phân **Protobuf (Protocol Buffers)** để đẩy qua Redis và Frontend (nhỏ và nhanh hơn JSON rất nhiều).
- **Quản lý kết nối Client:** Lấy dữ liệu từ Redis Pub/Sub và "bơm" (fan-out) tới hàng ngàn tab trình duyệt của người dùng đang online thông qua WebSocket.

3. BACKEND PYTHON: Lớp Toán học Định lượng & Động cơ AI

Nhiệm vụ: Chạy ngầm, làm các phép toán ma trận phức tạp và gọi DeepSeek để ra quyết định.

- **Framework: FastAPI** (nhanh, hỗ trợ bất đồng bộ `asyncio` và tích hợp sẵn Pydantic).
- **Động cơ Tính toán Định lượng (Quant Engine):**
    - **DuckDB:** Cơ sở dữ liệu chạy thẳng trên RAM (In-memory). Dùng để tính toán POC, VAH, VAL (của TPO) và CVD cực kỳ nhanh từ dữ liệu thô. (Kế thừa kiến trúc từ dự án `cryexc-backend`).
    - **NumPy / Pandas:** Xử lý ma trận dữ liệu mảng.
- **AI & Khớp lệnh Schema:**
    - **Pydantic:** Đây là "vũ khí tối thượng". Bạn định nghĩa một Schema chặt chẽ cho Kế hoạch Giao dịch (Entry, SL, TP, R:R).
    - **DeepSeek V4.1 Flash API (Function Calling / Structured Outputs):** Gửi dữ liệu thị trường và ép DeepSeek phải trả về đúng định dạng Pydantic Schema. Nếu AI trả về sai định dạng chữ hoặc tính sai toán học, Pydantic sẽ báo lỗi và từ chối.
- **Bộ nhớ Dài hạn cho AI (RAG): ChromaDB**. Lưu lại lịch sử các phán đoán đúng/sai của AI để những lần phân tích sau AI tự rút kinh nghiệm (tránh lặp lại sai lầm).

4. DATABASE: Lớp Lưu Trữ Cốt Lõi

Không dùng 1 DB cho tất cả. Chúng ta chia làm 2 loại:

- **Hot-Cache & Message Broker: Redis** (hoặc Valkey).
    - Lưu trạng thái giá hiện tại, nến hiện tại.
    - Làm kênh Pub/Sub kết nối giữa Backend Go và Backend Python.
- **Time-Series Database (Lưu trữ lịch sử): PostgreSQL** với extension **TimescaleDB**.
    - Lưu trữ nến lịch sử (1m, 15m, 1H), dữ liệu Funding, OI.

- Lưu trữ lịch sử tín hiệu của AI (để làm bảng xếp hạng Win-rate, ROI cho website).

Quy Trình Dòng Chảy (Tích hợp TanStack)

1. **Dữ liệu vào:** Go hứng dữ liệu thô từ sàn → Đẩy vào Redis.
2. **Tính toán AI:** Python lấy dữ liệu từ Redis → Dùng DuckDB tính TPO/CVD → Gửi prompt cho DeepSeek → Lưu JSON kết quả (Trade Setup) vào PostgreSQL → Báo tín hiệu có lệnh mới lên Redis.
3. **Dữ liệu ra Frontend:** Go nhận lệnh mới từ Redis → Bắn tín hiệu WebSocket cho Frontend.
4. **Frontend (React + TanStack):**
   - Luồng Web Worker nhận dữ liệu, vẽ lại nến và Footprint trên `lightweight-charts`.
   - **TanStack Query** nhận tín hiệu có Setup AI mới → cập nhật ngay lập tức bộ đệm cục bộ (cache) để **TanStack Table** (ở cột Radar bên phải) tự động hiển thị dòng tín hiệu mới màu xanh/đỏ mà không hề giật trang.

Với tech stack này, bạn có một hệ thống mạnh ngang ngửa các công cụ chuyên nghiệp trên thị trường (như Coinalyze, Exocharts) mà AI thì thông minh hơn.

Bạn muốn đi sâu vào code của phần nào trước: **Khởi tạo Frontend (React + TanStack + Lightweight Charts)** hay **Xây dựng Go Gateway hứng dữ liệu sàn**?

User prompt: roi kien truc tong quan cua repo nhu nao chia thanh 2 project tach biet frontend va backend roi tach rieng ra cac project rieng cho ai va phan con lai cua backend laf 3 repo hay sao

Response: Tư duy phân tách của bạn hoàn toàn chính xác. Đối với một hệ thống sử dụng 3 ngôn ngữ lập trình khác nhau (JavaScript/TypeScript, Go, và Python) phục vụ các mục đích hoàn toàn trái ngược nhau, việc áp dụng **Kiến trúc Vi dịch vụ (Microservices)** và chia thành **3 Repository (Repo) riêng biệt** là giải pháp tối ưu nhất, chuẩn theo môi trường công nghiệp.

Dưới đây là cấu trúc tổng quan chi tiết cho 3 Repo này, nhiệm vụ của từng cái và cách chúng giao tiếp với nhau.

1. Repo 1: `orderflow-frontend` (Lớp Trình diễn)

Đây là dự án Next.js (hoặc Vite React) chịu trách nhiệm hiển thị giao diện, biểu đồ và tương tác với người dùng.

- **Tech Stack:** Next.js, TypeScript, Tailwind CSS, TanStack Query, Lightweight Charts.
- **Đặc điểm triển khai:** Triển khai tĩnh (Static) trên Vercel, Netlify hoặc CDN. Không xử lý logic kinh doanh nặng, chỉ tập trung vào tốc độ render (60 FPS) và quản lý bộ nhớ đệm (cache) cho trình duyệt.
- **Cấu trúc thư mục cốt lõi:**

```
orderflow-frontend/
├── src/
│   ├── components/
│   │   ├── charts/       # Canvas components (K-line, Footprint, TPO)
│   │   ├── radar/        # AI Opportunity UI, TanStack Tables
│   │   └── ui/           # Nút bấm, Modal, Layout (Shadcn UI)
│   ├── hooks/            # Custom hooks (useWebSocket, useTanStackQuery)
│   ├── workers/          # Web Workers (giải nén dữ liệu WS ngầm, tránh lag UI)
│   ├── store/            # Zustand (quản lý state cục bộ)
│   └── services/         # File kết nối API & WebSocket tới Backend
├── package.json
└── tailwind.config.js
```

2. Repo 2: `orderflow-gateway-go` (Lớp Truyền tải Thời gian thực)

Đây là "trái tim" về mặt hạ tầng mạng. Repo Go này chỉ làm 2 việc nhưng làm với tốc độ siêu nhanh (độ trễ tính bằng mili-giây): Hứng dữ liệu từ sàn và bơm dữ liệu cho trình duyệt người dùng.

- **Tech Stack:** Golang, Fiber/Gin, Gorilla WebSocket, Go-Redis.
- **Đặc điểm triển khai:** Triển khai trên Docker Container (VPS). Có thể nhân bản (scale) lên nhiều server dễ dàng nếu lượng người dùng xem biểu đồ tăng lên hàng ngàn người.
- **Cấu trúc thư mục cốt lõi:**

```
orderflow-gateway-go/
├── cmd/
│   └── server/           # Điểm khởi chạy (main.go)
├── internal/
```

```
│   ├── exchange/         # Logic kết nối WS tới Binance/Bybit (lấy @aggTrade, @depth)
│   ├── pubsub/           # Giao tiếp với Redis (Publish/Subscribe)
│   ├── client_ws/        # Quản lý hàng ngàn kết nối WS từ người dùng trình duyệt
│   └── models/           # Định nghĩa cấu trúc dữ liệu Protobuf / Struct
├── pkg/                  # Các tiện ích dùng chung (Logger, Config)
├── go.mod
└── Dockerfile
```

3. Repo 3: `orderflow-ai-python` (Lớp Toán học & Trí tuệ AI)

Đây là "bộ não" của hệ thống. Nó không quan tâm đến việc người dùng đang truy cập website như thế nào. Nó chỉ chạy ngầm, nhận dữ liệu thị trường từ Redis, tính toán các công thức phức tạp và gọi DeepSeek.

- **Tech Stack:** Python, FastAPI, DuckDB, Pydantic, ChromaDB.
- **Đặc điểm triển khai:** Triển khai trên Docker Container (VPS). Cần cấu hình phần cứng ưu tiên CPU và RAM mạnh vì DuckDB chạy trên RAM và Python xử lý ma trận toán học.
- **Cấu trúc thư mục cốt lõi:**

```
orderflow-ai-python/
├── main.py               # Khởi chạy FastAPI server (nhận trigger/cronjob)
├── core/
│   ├── quant_engine/     # Logic DuckDB tính TPO (POC, VAH), CVD Divergence
│   ├── ai_radar/         # Scripts kết nối API DeepSeek V4.1 Flash
│   └── prompts/          # Chứa các System Prompts ép khuôn DeepSeek
├── schemas/              # Pydantic models (ép định dạng JSON: Entry, SL, TP)
├── database/
│   ├── redis_client.py   # Lắng nghe dữ liệu mới & lưu tín hiệu AI
│   ├── timescaledb.py    # Lưu trữ lịch sử tín hiệu và nến 15m/1H
│   └── vectordb/         # ChromaDB (lưu trữ RAG kinh nghiệm giao dịch)
├── requirements.txt
└── Dockerfile
```

Luồng Giao Tiếp Giữa 3 Repo (Cách chúng ghép lại thành một khối)

Để 3 dự án độc lập này hoạt động đồng nhất, bạn cần một **Message Broker (Trạm trung chuyển)**. Trong trường hợp này, **Redis** là cầu nối hoàn hảo.

1. **Go → Python (Bơm dữ liệu thô):** Go hứng lệnh từ Binance, đóng gói gọn lại và lưu vào Redis.
2. **Python → Redis (Phát tín hiệu AI):** Python (chạy ngầm định kỳ hoặc theo sự kiện) lấy dữ liệu từ Redis, tính toán, gọi DeepSeek. Khi ra tín hiệu "MUA BTC", Python đẩy tín hiệu JSON này vào một kênh Redis (ví dụ: `channel:ai_signals`).
3. **Redis → Go → Frontend (Bắn kết quả cho người dùng):**
   - Repo Go lắng nghe kênh `channel:ai_signals` từ Redis.
   - Ngay khi có tín hiệu, Go chuyển tiếp qua WebSocket bắn thẳng tới Frontend.
   - Frontend (TanStack Query) nhận được, lập tức vẽ bảng tín hiệu và hộp Entry/SL/TP lên biểu đồ.

Việc chia thành 3 Repo mang lại lợi thế khổng lồ: Nếu bạn bảo trì, nâng cấp, hoặc viết lại hệ thống Prompts AI trong Python, hệ thống biểu đồ thời gian thực (Go + Frontend) của người dùng vẫn hoạt động trơn tru mà không bị sập hay gián đoạn.

User prompt: roi no giao tiep bang gi grtc hay api hay gi

Response: Câu hỏi của bạn rất chuẩn xác. Trong kiến trúc Microservices (Vi dịch vụ) xử lý dữ liệu tài chính, chúng ta **không dùng một giao thức duy nhất** mà phải kết hợp dựa trên tính chất của luồng dữ liệu (Đồng bộ hay Bất đồng bộ).

Để hệ thống hoạt động với độ trễ thấp nhất và không làm "sập" lẫn nhau khi quá tải, 3 dự án này sẽ giao tiếp qua **3 cây cầu chính**:

1. Giữa Trình Duyệt (Next.js) ↔ Go Gateway

*Giao thức: **WebSocket (WSS) + REST API (HTTP/2)***

- **REST API:** Chỉ dùng một lần duy nhất lúc người dùng mới mở trang web (Page Load). Frontend sẽ gọi API để kéo một cục dữ liệu lớn (Ví dụ: Dữ liệu nến 1 tháng qua, hoặc 50 tín hiệu AI gần nhất). TanStack Query sẽ nhận dữ liệu này và lưu vào bộ đệm (cache).
- **WebSocket:** Ngay sau khi kéo REST API xong, Frontend mở kết nối WebSocket với Go. Từ lúc này, Go sẽ "bơm" dữ liệu liên tục (giá tick mới, cột footprint mới, tín hiệu AI vừa chốt) thẳng vào trình duyệt mà không cần Frontend phải hỏi.

2. Giữa Go ↔ Python (Luồng Bất Đồng Bộ - Luồng chính)

Giao thức: ***Redis Pub/Sub (hoặc Redis Streams)***

Đây là xương sống của hệ thống. Bạn **không nên dùng gRPC hay REST API** để Go nhồi dữ liệu thị trường sang Python.

- **Lý do:** Go hứng 10.000 giao dịch mỗi giây từ sàn. Nếu Go dùng gRPC gọi sang Python 10.000 lần/giây, server Python sẽ treo ngay lập tức vì Python xử lý đồng thời (concurrency) kém hơn Go rất nhiều.
- **Cách hoạt động (Decoupling):**
  - Go gom dữ liệu từ sàn và cứ thế "ném" vào một kênh của Redis (Ví dụ: `channel:btc_trades`). Go không quan tâm Python có đang rảnh hay không.
  - Python sẽ "đăng ký" (Subscribe) kênh này. Nó âm thầm gom dữ liệu trong bộ nhớ DuckDB. Cứ mỗi 15 phút, nó tính toán ma trận TPO, gọi DeepSeek, và ném kết quả (Trade Setup) vào một kênh Redis khác (`channel:ai_signals`).
  - Go đang chực chờ ở kênh `ai_signals`, thấy có tín hiệu AI mới là lập tức chộp lấy và bắn qua WebSocket cho người dùng.

3. Giữa Go ↔ Python (Luồng Đồng Bộ - Ra Lệnh)

Giao thức: ***gRPC (dùng Protobuf)***

Mặc dù luồng chính dùng Redis, nhưng sẽ có những lúc Go cần ra lệnh trực tiếp cho Python và cần kết quả ngay lập tức (Ví dụ: Người dùng bấm nút *"Yêu cầu Radar quét lại đồng ETH ngay bây giờ"*).

- **Cách hoạt động:** Thay vì dùng REST API nội bộ (chậm và tốn tài nguyên do phải parse JSON), Go và Python sẽ giao tiếp với nhau bằng **gRPC**.
- **Lý do:** gRPC truyền tải dữ liệu bằng mã hóa nhị phân (Protocol Buffers - Protobuf) qua HTTP/2. Tốc độ giao tiếp nội bộ giữa 2 microservice bằng gRPC **nhanh gấp 7 đến 10 lần** so với việc dùng REST API thông thường. Cả Go và Python đều hỗ trợ Protobuf cực kỳ mạnh mẽ.

Tóm tắt lại Bản Đồ Giao Tiếp:

1. Frontend hỏi xin lịch sử cũ → **REST API** → Go
2. Go bắn dữ liệu realtime cho Frontend → **WebSocket**
3. Cập nhật dòng chảy giá & Kế hoạch AI giữa Go và Python → **Redis Pub/Sub**
4. Go gọi Python thức dậy xử lý tác vụ đột xuất → **gRPC**

Nếu bạn đồng ý với luồng giao tiếp này, chúng ta có thể chốt kiến trúc. Bạn muốn bắt tay vào việc **định nghĩa các file Protobuf (.proto)** để làm chuẩn giao tiếp giữa Go và Python, hay muốn bắt đầu **cài đặt khung dự án Go Gateway** trước?

User prompt: chot lai kien truc truoc di

Response: Để đảm bảo chúng ta đi đúng hướng và không phải đập đi xây lại sau này, đây là **Bản Chốt Kiến Trúc Tổng Thể (Final Architecture Blueprint)**.

Hệ thống được thiết kế theo chuẩn Vi dịch vụ (Microservices), chia thành 3 dự án (Repository) độc lập hoàn toàn về mặt logic và ngôn ngữ, giao tiếp với nhau qua trạm trung chuyển Redis và gRPC.

1. Thành Phần Cốt Lõi (Các Repository)

Repo 1: Lớp Giao Diện (orderflow-frontend)

- **Vai trò:** Nhận dữ liệu và vẽ biểu đồ siêu tốc (60 FPS), hiển thị Radar AI, không xử lý logic tính toán nặng.
- **Tech Stack:** Next.js (hoặc Vite React), TypeScript, Tailwind CSS.
- **Thư viện lõi:**
  - **TanStack Query & Table:** Quản lý bộ nhớ đệm API và bảng dữ liệu Radar mượt mà.
  - **TradingView Lightweight Charts + plugin `lightweight-orderflow-charts`:** Vẽ nến, TPO, Footprint, và CVD.
- **Tối ưu:** Dùng HTML5 Web Workers để giải nén dữ liệu WebSocket ngầm, giữ cho tab trình duyệt không bị treo.

Repo 2: Cổng Dữ Liệu Thời Gian Thực (orderflow-gateway-go)

- **Vai trò:** Chịu tải mạng. Hứng hàng chục nghìn tick từ sàn giao dịch, đóng gói, và phân phối đến hàng nghìn tab trình duyệt của người dùng.
- **Tech Stack:** Golang, Fiber (hoặc Gin), Gorilla WebSocket.
- **Quy trình:** Lắng nghe Binance/Bybit WS → Gom nhóm lệnh (Batching mỗi 100-250ms) → Đóng gói bằng Protobuf → Ghi vào Redis → Bắn WSS cho Frontend.

Repo 3: Động Cơ Toán Học & AI (orderflow-ai-python)

- **Vai trò:** Khối óc của hệ thống. Chạy ngầm, đọc dữ liệu dòng lệnh, tính toán hỗ trợ/kháng cự vi mô, và gọi AI để ra tín hiệu giao dịch.
- **Tech Stack:** Python, FastAPI.
- **Bộ tính toán (Quant):** Sử dụng **DuckDB** (chạy trên RAM) để tính Value Area (VAH, VAL, POC) và Phân kỳ CVD cực nhanh từ dữ liệu thô.
- **Bộ phận AI:** Gọi API **DeepSeek V4.1 Flash**, dùng **Pydantic** để ép AI phải trả về chuẩn JSON (Entry, Stop Loss, Take Profit, R:R). Tích hợp **ChromaDB** để lưu trữ RAG kinh nghiệm đánh lệnh.

2. Hệ Thống Lưu Trữ (Database Layer)

- **Redis (Hot Cache & Message Broker):**
  - Đóng vai trò "Kênh truyền tin" (Pub/Sub) giữa Go và Python.
  - Lưu trữ trạng thái giá hiện tại và các tín hiệu AI vừa chốt để tải nhanh cho người dùng mới vào web.
- **PostgreSQL + TimescaleDB (Cold Storage):**
  - Lưu trữ vĩnh viễn dữ liệu nến (OHLCV) các khung thời gian lớn, dữ liệu Funding Rate, Open Interest (từ API CoinGlass).
  - Lưu lại lịch sử các Kế hoạch Giao dịch (Trade Setups) do AI tạo ra để làm bảng xếp hạng tỷ lệ Win/Loss.

3. Bản Đồ Giao Tiếp Giữa Các Khối (Data Flow)

Kiến trúc này giải quyết triệt để nút thắt cổ chai về hiệu năng bằng cách sử dụng đúng giao thức cho đúng việc:

1. **Go → Frontend:** Kết nối bằng **WebSocket**. Go chủ động đẩy dữ liệu giá và tín hiệu Radar mới nhất lên trình duyệt.
2. **Frontend ↔ Trạm Dữ liệu:** Khi người dùng vừa mở web hoặc đổi khung thời gian, Frontend gọi **REST API** (qua TanStack Query) để lấy dữ liệu lịch sử từ PostgreSQL.
3. **Go → Python (Luồng nạp dữ liệu):** Kết nối bất đồng bộ qua **Redis Pub/Sub**. Go ném dữ liệu thô vào Redis, Python tự động lấy về nhồi vào DuckDB để tính toán.
4. **Python → Trình duyệt (Luồng phát tín hiệu):** Khi DeepSeek ra một tín hiệu đẹp → Python kiểm tra R:R → Lưu vào PostgreSQL → Bắn thông báo lên Redis → Go nhận được và đẩy WSS thẳng tới Frontend để vẽ hộp Entry/SL/TP lên biểu đồ.
5. **Go ↔ Python (Luồng đồng bộ):** Khi hệ thống cần Go ra lệnh trực tiếp cho Python (VD: Người dùng bấm "Quét ép buộc"), hai bên giao tiếp nội bộ bằng **gRPC / Protobuf** để đạt tốc độ tối đa.

Đây là cấu trúc hoàn chỉnh, sẵn sàng chịu tải chuẩn công nghiệp và hoàn toàn khả thi để xây dựng theo từng giai đoạn.

User prompt: ok dựa vào những gì đã bàn ở trên t cần research lại những opensource nào mình có thể dùng cho các phần trên của mình có thể theo những pattern của họ để có thể giải quyết được vấn đề của mình về design rồi tới xử lí logic làm sao để tăng độ chính xác rồi xử lí dữ liệu rồi xử lí các thứ như nào t cần một bản trình bày kết quả chi tiết

Response: Dựa trên báo cáo nghiên cứu chuyên sâu về 42 dự án mã nguồn mở trong lĩnh vực High-Frequency Trading (HFT), Order Flow và AI Agents, đây là **Bản Thiết Kế Kỹ Thuật & Kế Thừa Mã Nguồn Mở Toàn Diện** được ánh xạ trực tiếp vào kiến trúc **3 Repositories (`Frontend Next.js/TanStack` - `Gateway Go` - `AI Engine Python`)** của bạn.

Mục tiêu của bản thiết kế này là chỉ rõ: **Ở mỗi Repo, chúng ta lấy chính xác đoạn code/thuật toán nào từ mã nguồn mở nào, áp dụng Design Pattern gì để xử lý dữ liệu và làm thế nào để tăng độ chính xác của DeepSeek V4.1 Flash.**

PHẦN 1: Bảng Ánh Xạ Mã Nguồn Mở Vào 3 Repositories

Repository của bạn	Dự án Mã nguồn mở kế thừa	Cái chúng ta lấy trực tiếp (Code / Thư viện)	Design Pattern học hỏi từ họ
1. `orderflow-frontend` *(Next.js + TanStack + Lightweight Charts)*	• `awahab07/lightweight-orderflow-charts` • `tapedelta/kline-orderbook-chart` • `Tucsky/aggr`	• Bộ vẽ Footprint, TPO, POC và Delta Summary. • Lớp phủ Liquidation Heatmap. • Web Worker gom lệnh (Batching).	**Dual-Surface Formatting & Timeout Aggregation:** Tách luồng nhận WS sang Web Worker, gom lệnh mỗi 200ms rồi mới đẩy vào bộ đệm của `TanStack Query`.

Repository của bạn	Dự án Mã nguồn mở kế thừa	Cái chúng ta lấy trực tiếp (Code / Thư viện)	Design Pattern học hỏi từ họ
2. `orderflow-gateway-go` *(Golang + Fiber + Gorilla WS + Redis)*	• `edgedepthhq/edgedepth-gateway` • `apache/superset-websocket` • `Tucsky/aggr-server`	• Kiến trúc Go thu thập `@aggTrade`, `@depth` và đóng gói Protobuf. • Cơ chế Redis Pub/Sub fan-out. • Cơ chế nối dữ liệu RAM + DB.	**Pluggable Venue Interface & Envelope Pattern:** Chuẩn hóa dữ liệu đa sàn + CoinGlass về một struct duy nhất; dùng Zstd/Protobuf giảm 70% băng thông.
3. `orderflow-ai-python` *(FastAPI + DuckDB + DeepSeek + ChromaDB)*	• `jose-donato/cryexc-backend` • `wickra-lib/wickra` • `ymys/Bot-Auto-Screening-Bybit` • `qrak/LLM_trader` • `anvilfilbert/Auto-Crypto-Tradingjournal`	• DuckDB In-Memory tính CVD/Footprint. • Thuật toán 2 con trỏ tính VAH/VAL/POC. • Bộ chấm điểm CVD Divergence. • Kiến trúc RAG ChromaDB + Claim Validation. • Quản lý vòng đời lệnh (Multi-TP, Drift Guard).	**Deterministic Pre-Scoring + Agentic RAG + Pydantic Guardrail:** Không để AI đoán mò; dùng toán học chấm điểm trước, lấy lịch sử thắng/thua từ ChromaDB, và khóa đầu ra bằng Pydantic.

PHẦN 2: Giải Quyết Bài Toán Thiết Kế & Hiển Thị (`orderflow-frontend`)

Thách thức lớn nhất ở Frontend là hiển thị cùng lúc K-line, TPO, Footprint, Liquidation Heatmap và bảng tín hiệu AI mà không làm treo trình duyệt hoặc lệch lưới giá.

1. Kế thừa cấu trúc `OrderFlowBar` và `Mintick Bucketing` (từ `lightweight-orderflow-charts`)

- **Vấn đề thực tế:** Dữ liệu từ sàn gửi về thường bị "thưa" (sparse data) — tức là có những mức giá không có giao dịch nào khiến các ô Footprint bị lệch hàng khi người dùng phóng to/thu nhỏ biểu đồ.
- **Giải pháp áp dụng:** Sử dụng hàm helper `clusterOrderFlowBarsByMintick` của thư viện này. Nó tự động gom nhóm các mức giá theo bước giá nhỏ nhất (`mintick`), lấp đầy các mức giá trống để giữ cấu trúc hình học của Footprint và TPO luôn thẳng hàng tuyệt đối với nến K-line.
- **Định dạng Bề mặt Kép (Dual-Surface Text Formatting):** Tách biệt hiển thị trục giá (ví dụ `priceDecimalDigits: 2` cho ra `64,250.50`) và chữ số bên trong ô Footprint (`valueUnitVisible: true` rút gọn khối lượng `125,400` thành `125.4k`) để giao diện không bị tràn chữ.

2. Pattern "Web Worker + TanStack Query Cache Mutation" (học từ `Tucsky/aggr`)

- **Cách `aggr` xử lý:** Không bao giờ đẩy từng lệnh khớp (tick) thẳng vào giao diện. Họ dùng một bộ đệm thời gian (Timeout-based aggregation) gom hàng trăm lệnh có cùng mili-giây lại.
- **Cách tích hợp với TanStack của bạn:**
  1. **Luồng biểu đồ (60 FPS):** Web Worker nhận luồng nhị phân (Protobuf) từ Go Gateway, gom lệnh trong `200ms`, sau đó bắn thẳng vào hàm `series.update()` của Lightweight Charts (bỏ qua vòng đời re-render của React).
  2. **Luồng AI Opportunity Radar (TanStack Query + TanStack Table):** Khi có tín hiệu AI mới hoặc cập nhật trạng thái lệnh (`PENDING` → `ACTIVE` → `HIT_TP`), WebSocket đẩy JSON sự kiện về → gọi trực tiếp `queryClient.setQueryData(['ai-radar-signals'], updater)`. `TanStack Table` sẽ tự động cập nhật đúng dòng tín hiệu đó với hiệu ứng nhấp nháy mà không cần tải lại danh sách.

PHẦN 3: Giải Quyết Bài Toán Xử Lý Luồng Dữ Liệu Tốc Độ Cao (`orderflow-gateway-go`)

Ở tầng Go, chúng ta kết hợp dữ liệu từ **CoinGlass Standard API ($299/tháng)** và **WebSocket trực tiếp của sàn (Binance/Bybit)**.

1. Kiến trúc `Pluggable Venues` & Nén `Protobuf/Zstd` (từ `edgedepth-gateway`)

- Dự án `edgedepth-gateway` (viết bằng Go) cung cấp sẵn mẫu thiết kế tách biệt:
  - **Luồng Tốc độ cao (WebSocket Sàn):** Hứng `@aggTrade` (lệnh khớp mua/bán chủ động để tính Volume Delta & CVD) và `@depth@100ms`.
  - **Luồng Thăm dò Định kỳ (REST Poller cho CoinGlass API):** Cứ mỗi 10–30 giây, một Goroutine gọi CoinGlass API lấy **Open Interest (OI), Funding Rate, và Liquidation Levels/Heatmap**.
- **Tối ưu hóa truyền tải:** Thay vì bắn chuỗi JSON nặng nề qua mạng, Go đóng gói dữ liệu thành **Protobuf** kèm cơ chế nén **Zstd** (giống hệt `edgedepth-gateway`), giảm tới 70% băng thông mạng từ VPS về trình duyệt.

2. Pattern "Hybrid Hot-Buffer + Cold-DB" (từ `aggr-server` & `superset-websocket`)

- **Vấn đề:** Khi người dùng mới mở trang web, dữ liệu lịch sử đang nằm trong PostgreSQL (TimescaleDB), nhưng 5 giây dữ liệu mới nhất lại đang nằm trên RAM và chưa kịp ghi xuống ổ cứng. Nếu chỉ lấy từ DB, biểu đồ sẽ bị hụt mất vài cây nến cuối.
- **Giải pháp từ `aggr-server`:** Khi Frontend gọi REST API lấy lịch sử nến, Go Gateway sẽ truy vấn TimescaleDB kết hợp nối (merge) trực tiếp với bộ đệm chưa ghi (unflushed buffer) trên Redis, trả về một mạch dữ liệu liền lạc 100%.
- **Cơ chế `Message Envelope` & `Connection Draining` (từ `superset-websocket`):** Mọi gói tin đẩy qua Redis Pub/Sub đều được bọc trong một cấu trúc chuẩn gồm `topic` (ví dụ: `orderflow:btcusdt:1m` hoặc `radar:signal:new`), `scope`, và `payload`. Nếu kết nối Redis gặp sự cố, endpoint `/ready` của Go sẽ tự động báo để ngắt điều hướng, đảm bảo người dùng không bao giờ nhìn thấy dữ liệu bị đứng hình mà không hay biết.

PHẦN 4: Xử Lý Logic Toán Học & Tăng Độ Chính Xác Cho AI (`orderflow-ai-python`)

Đây là phần quan trọng nhất mà bạn quan tâm: **Làm sao để xử lý dữ liệu thô và khiến DeepSeek V4.1 Flash đưa ra quyết định giao dịch có độ chính xác cao nhất?**

Câu trả lời từ các dự án mã nguồn mở là: **Không bao giờ để LLM tự làm toán và tự đoán mô hình giá.** Chúng ta phải áp dụng quy trình **5 Lớp Lọc Chính Xác (5-Layer Accuracy Pipeline)** dưới đây:

1. **Lớp 1: Tính toán In-Memory siêu tốc với DuckDB**

   Học từ dự án cryexc-backend

   Thay vì dùng vòng lặp Python chậm chạp hay truy vấn ổ cứng liên tục, service Python nhét toàn bộ luồng giao dịch 24h gần nhất từ Redis vào **DuckDB chạy trên RAM**.

   - Sử dụng khả năng xử lý vector của DuckDB để gộp hàng triệu tick thành bảng `Footprint` (nhóm theo khung thời gian 15m/1H/4H và bước giá `price_step`) cùng chuỗi **CVD (Cumulative Volume Delta)** chỉ trong vài mili-giây.
   - Tự động chạy cronjob mỗi giờ xóa dữ liệu quá 24h trong DuckDB để RAM luôn nhẹ.

2. **Lớp 2: Thuật toán Hai Con Trỏ (Two-Pointer) tìm chuẩn xác POC, VAH, VAL**

   Học từ dự án wickra-lib & PineScript TPO

   Trước khi gọi AI, Python chạy thuật toán chuẩn xác để xác định vùng giá trị TPO/Volume Profile:

   1. Tìm mức giá có tổng khối lượng/thời gian lớn nhất làm điểm neo **POC** ($pocIndex$) và tính tổng khối lượng toàn phiên ($TotalVolume$).
   2. Đặt 2 con trỏ: $upIndex = pocIndex + 1$ và $downIndex = pocIndex - 1$.
   3. So sánh khối lượng tại $upIndex$ ($volUp$) và $downIndex$ ($volDown$). Bên nào lớn hơn thì cộng vào biến tích lũy $volumeAccum$ và dịch con trỏ đó ra xa thêm 1 nấc.
   4. Lặp lại cho đến khi $volumeAccum \geq 0.70 \times TotalVolume$. Vị trí dừng của 2 con trỏ chính là **VAH (Value Area High)** và **VAL (Value Area Low)** chính xác tuyệt đối về mặt toán học.

3. **Lớp 3: Bộ Chấm Điểm Định Lượng (Deterministic Pre-Scoring) & Lọc Bẫy**

   Học từ dự án Bot-Auto-Screening-Bybit (bybit_bot_v8)

   Thay vì gửi mọi cây nến cho DeepSeek, chúng ta áp dụng bộ lọc chấm điểm cứng của `bybit_bot_v8` để loại bỏ 80% tín hiệu nhiễu (Fakeouts):

   - **Phát hiện Phân kỳ CVD + OBV làm mượt (EMA 20):** So sánh đỉnh/đáy của Giá với đỉnh/đáy của CVD. Nếu Giá tạo Đáy Thấp Hơn (Lower Low) nhưng CVD tạo Đáy Cao Hơn (Higher Low) $\rightarrow$ Ghi nhận **Regular Bullish Divergence** (người bán chủ động đã kiệt sức, lệnh Limit Buy đang hấp thụ hết) $\rightarrow$ Cộng **+2 điểm**.
   - **Bảo vệ chống Phá vỡ giả (Fakeout Protection):** Tính Khối lượng Tương đối ($RVOL$). Nếu $RVOL$ nhỏ hơn 2.0 trong một pha bứt phá $\rightarrow$ **Hủy tín hiệu ngay lập tức** (không gọi AI). Nếu $RVOL$ lớn hơn 5.0 (Nuclear RVOL) $\rightarrow$ Cộng **+1 điểm**.

- **Lọc Bẫy (Trap Detection):** Nếu tín hiệu báo Long nhưng RSI đang lớn hơn 75 hoặc Funding Rate từ CoinGlass đang quá dương (đám đông đang fomo Long) → Đánh dấu **Bullish Exhaustion Trap** và loại bỏ.

4 **Lớp 4: Tăng cường Trí nhớ Dài hạn (Agentic RAG với ChromaDB) & Chống Ảo Giác**

Học từ dự án qrak/LLM_trader

Khi một cơ hội vượt qua Lớp 3 (điểm định lượng cao), hệ thống mới gọi **DeepSeek V4.1 Flash**. Để tăng độ chính xác của DeepSeek, chúng ta áp dụng 2 kỹ thuật đỉnh cao từ `LLM_trader`:

- **Brain with Memory (ChromaDB RAG):** Trước khi gửi prompt, Python dùng mô hình nhúng `bge-base-en-v1.5` tìm trong ChromaDB và đính kèm vào prompt:
  1. **3 lệnh giao dịch trong quá khứ** có cấu trúc Order Flow (CVD, OI, TPO) giống hệt hiện tại nhất và kết quả thắng/thua của chúng.
  2. **Win-rate của 20 lệnh gần nhất** và **5 bài học xương máu (post-mortem lessons)** từ các lệnh bị cắt lỗ trước đó để ép AI không lặp lại sai lầm cũ.
- **Claim Validation (Kiểm chứng chéo chống AI nói bừa):** Nếu trong phần giải thích, DeepSeek tuyên bố *"Xu hướng hiện tại cực kỳ mạnh"*, code Python sẽ tự động đối chiếu với chỉ số ADX và OI thực tế. Nếu ADX thực tế đang thấp (thị trường đi ngang), hệ thống tự động hạ điểm tự tin (`confidence_score`) của AI xuống.

5 **Lớp 5: Khóa Pydantic Schema & Quản lý Vòng đời Lệnh (Lifecycle Guard)**

Học từ dự án crypto-signals & Auto-Crypto-Tradingjournal

- **Ép khuôn bằng Pydantic:** Bắt buộc DeepSeek trả về đúng cấu trúc JSON chứa: `trigger_condition`, `invalidation_condition`, `entry_price`, `stop_loss`, `tp1`, `tp2`, `tp3`.
- **Kiểm tra Tỷ lệ R:R Cứng (Hard-coded R:R Check):** Code Python tự tính lại:

$$R:R = \frac{|TP - Entry|}{|Entry - SL|}$$

  Nếu $R:R$ nhỏ hơn 2.0 (đối với Intraday) hoặc nhỏ hơn 3.0 (đối với HTF), hệ thống **tự động bác bỏ lệnh của AI**.
- **Entry-Drift Guard & Multi-TP Ladder:** Khi tín hiệu đã phát lên web, Python theo dõi giá realtime:
  - Nếu giá chạy quá xa vùng Entry (lệch quá 2%) trước khi khớp điều kiện kích hoạt, hoặc chạm `invalidation_condition` → Tự động đổi trạng thái trên web thành `INVALIDATED` (Hủy kèo).
  - Chia mục tiêu chốt lời thành 3 bậc (`TP1: 40%`, `TP2: 40%`, `TP3: 20%`). Ngay khi giá chạm `TP1`, hệ thống tự động cập nhật giao diện báo hiệu **"Đã dời Stop Loss về Hòa Vốn (Break-Even + 0.25% phí)"**.

PHẦN 5: Tổng Hợp Luồng Xử Lý Dữ Liệu Để Tăng Độ Chính Xác (Sơ Đồ Tóm Tắt)

Để bạn dễ hình dung bức tranh tổng thể khi bắt đầu code, mọi dữ liệu đi vào hệ thống sẽ chảy qua phễu lọc 4 bước sau:

1. **Dữ liệu Thô (Raw Data):** CoinGlass API (OI, Funding, Liquidations) + Sàn WS (Trades, Orderbook) → **Go Gateway** gom cụm mỗi 200ms → bắn vào **Redis**.
2. **Tinh chế Toán học (Quant Refinement): Python + DuckDB** đọc từ Redis → tính chính xác `POC`, `VAH`, `VAL`, `CVD`, `RVOL`, `OI Delta %`.
3. **Lọc Nhiễu & Bơm Ký Ức (Pre-Filter & RAG):** Thuật toán `bybit_bot_v8` chấm điểm. Nếu đạt ngưỡng → Truy xuất **ChromaDB** lấy 3 kèo lịch sử tương đồng + bài học thất bại → Đóng gói thành **JSON State Vector (~500 tokens)**.
4. **Suy luận AI & Kiểm định (DeepSeek + Guardrail):** Gửi cho **DeepSeek V4.1 Flash** → Nhận về **Pydantic JSON** → Kiểm tra chéo toán học ($R:R \geq 2.0$, đối chiếu ADX/TPO) → Lưu vào **TimescaleDB** và đẩy qua **Go WebSocket** lên **TanStack Frontend**.

User prompt: Hãy viết code Python hoàn chỉnh cho Lớp 1, 2 và 3: Dùng DuckDB tính TPO (POC, VAH, VAL) và thuật toán chấm điểm phân kỳ CVD trước khi gọi AI.

Response: Dưới đây là mã nguồn Python hoàn chỉnh và tối ưu cho **3 Lớp đầu tiên (Layer 1, 2, và 3)** của hệ thống. Đoạn code này thực hiện việc nhận luồng giao dịch thô, xử lý bằng DuckDB trên RAM, tính toán Vùng Giá Trị (POC, VAH, VAL) bằng thuật toán Hai Con Trỏ (Two-Pointer), và chạy bộ lọc chấm điểm phân kỳ CVD trước khi quyết định có gọi DeepSeek AI hay không.

```
import duckdb
import numpy as np
import pandas as pd


class OrderFlowQuantEngine:

  def __init__(self):
```

```
        # Khởi tạo kết nối DuckDB chạy hoàn toàn trên RAM (In-Memory)
        self.conn = duckdb.connect(database=':memory:')
        self._init_schema()

    def _init_schema(self):
        """Khởi tạo bảng chứa dữ liệu giao dịch thô (Raw Trades)"""
        self.conn.execute("""
                CREATE TABLE IF NOT EXISTS raw_trades (
                    timestamp TIMESTAMP,
                    price DOUBLE,
                    size DOUBLE,
                    side VARCHAR -- 'BUY' hoặc 'SELL'
                );
            """)

    def ingest_trades(self, df_trades: pd.DataFrame):
        """Nạp dữ liệu giao dịch thô vào DuckDB"""
        self.conn.register('df_view', df_trades)
        self.conn.execute("""
                INSERT INTO raw_trades
                SELECT CAST(timestamp AS TIMESTAMP), price, size, side
                FROM df_view;
            """)
        # Xóa view tạm
        self.conn.unregister('df_view')

    # ==========================================================================
    # LỚP 1: IN-MEMORY PROCESSING VỚI DUCKDB (Tính Footprint & CVD)
    # ==========================================================================
    def layer_1_process_footprint_and_cvd(
        self, timeframe_minutes: int = 15, mintick: float = 1.0
    ):
        """Gom cụm dữ liệu thô thành các thanh Footprint (theo khung thời gian)

        và tính toán Cumulative Volume Delta (CVD) cực nhanh bằng DuckDB.
        """
        query = f"""
            WITH binned_trades AS (
                SELECT
                    -- Gom nhóm theo khung thời gian (ví dụ: 15 phút)
                    date_trunc('minute', timestamp) - INTERVAL (extract(minute from timestamp) % {timeframe_minute
                    -- Gom mức giá theo bước giá tối thiểu (mintick bucketing)
                    ROUND(price / {mintick}, 0) * {mintick} AS price_bin,
                    SUM(CASE WHEN side = 'BUY' THEN size ELSE 0 END) AS buy_volume,
                    SUM(CASE WHEN side = 'SELL' THEN size ELSE 0 END) AS sell_volume
                FROM raw_trades
                GROUP BY 1, 2
            ),
            delta_calc AS (
                SELECT
                    candle_time,
                    price_bin,
                    buy_volume,
                    sell_volume,
                    (buy_volume + sell_volume) AS total_volume,
                    (buy_volume - sell_volume) AS volume_delta
                FROM binned_trades
            )
            SELECT
                candle_time,
                price_bin,
                buy_volume,
                sell_volume,
                total_volume,
                volume_delta,
                -- Tính Cumulative Volume Delta (CVD) cộng dồn theo từng nến
                SUM(volume_delta) OVER (
                    PARTITION BY candle_time
                    ORDER BY price_bin
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
                ) AS cvd
            FROM delta_calc
            ORDER BY candle_time ASC, price_bin ASC;
        """
```

```
    return self.conn.execute(query).fetchdf()

  # ============================================================================
  # LỚP 2: THUẬT TOÁN HAI CON TRỎ (TWO-POINTER) TÍNH TPO (POC, VAH, VAL)
  # ============================================================================
  def layer_2_calculate_value_area(self, footprint_df: pd.DataFrame):
    """Tính toán Điểm Kiểm Soát (POC) và Vùng Giá Trị (VAH, VAL)

    dựa trên thuật toán Two-Pointer (chiếm 70% tổng khối lượng phiên).
    """
    if footprint_df.empty:
      return None

    # Nhóm tổng khối lượng theo từng mức giá trong toàn bộ tập dữ liệu truyền vào
    price_profile = (
        footprint_df.groupby('price_bin')['total_volume']
        .sum()
        .reset_index()
    )

    if price_profile.empty:
      return None

    # Sắp xếp giảm dần theo khối lượng để tìm POC (Mức giá có khối lượng lớn nhất)
    price_profile = price_profile.sort_values(
        by='total_volume', ascending=False
    ).reset_index(drop=True)

    poc_price = price_profile.loc[0, 'price_bin']
    total_volume = price_profile['total_volume'].sum()
    target_value_area_vol = total_volume * 0.70  # Chuẩn 70% Vùng Giá Trị

    # Sắp xếp lại tăng dần theo mức giá để thực hiện thuật toán Two-Pointer
    df_sorted = price_profile.sort_values(by='price_bin').reset_index(drop=True)
    poc_idx = df_sorted[df_sorted['price_bin'] == poc_price].index[0]

    # Khởi tạo 2 con trỏ quét ra xung quanh POC
    up_idx = poc_idx
    down_idx = poc_idx
    accumulated_vol = df_sorted.loc[poc_idx, 'total_volume']

    while accumulated_vol < target_value_area_vol:
      vol_up = (
          df_sorted.loc[up_idx + 1, 'total_volume']
          if (up_idx + 1) < len(df_sorted)
          else -1
      )
      vol_down = (
          df_sorted.loc[down_idx - 1, 'total_volume']
          if (down_idx - 1) >= 0
          else -1
      )

      # Mở rộng về phía có khối lượng lớn hơn
      if vol_up >= vol_down and vol_up != -1:
        up_idx += 1
        accumulated_vol += df_sorted.loc[up_idx, 'total_volume']
      elif vol_down != -1:
        down_idx -= 1
        accumulated_vol += df_sorted.loc[down_idx, 'total_volume']
      else:
        break

    val = float(df_sorted.loc[down_idx, 'price_bin'])
    vah = float(df_sorted.loc[up_idx, 'price_bin'])

    return {
        'POC': float(poc_price),
        'VAL': val,
        'VAH': vah,
        'TotalVolume': float(total_volume),
    }

  # ============================================================================
```

```
# LỚP 3: BỘ CHẤM ĐIỂM ĐỊNH LƯỢNG & PHÁT HIỆN PHÂN KỲ CVD
# ============================================================================
def layer_3_deterministic_prescore(
    self, price_lows: list, cvd_lows: list, rvol: float
):
  """Chạy bộ lọc định lượng cứng để phát hiện phân kỳ CVD và lọc bỏ bẫy

  trước khi cho phép gọi DeepSeek AI Radar.
  """
  score = 0
  signals = []

  # 1. Bộ lọc chống bẫy / Khối lượng tương đối (RVOL Filter)
  if rvol < 2.0:
    return {
        'status': 'REJECTED',
        'reason': (
            f'RVOL quá thấp ({rvol:.2f} < 2.0) - Nguy cơ phá vỡ giả (Fakeout)'
        ),
        'score': 0,
    }
  elif rvol >= 5.0:
    score += 1
    signals.append('Nuclear RVOL Spike (+1 điểm)')

  # 2. Thuật toán phát hiện Phân kỳ tăng giá (Bullish Divergence)
  # So sánh 2 điểm đảo chiều gần nhất: Giá tạo Đáy Thấp Hơn nhưng CVD tạo Đáy Cao Hơn
  if len(price_lows) >= 2 and len(cvd_lows) >= 2:
    price_lower_low = price_lows[-1] < price_lows[-2]
    cvd_higher_low = cvd_lows[-1] > cvd_lows[-2]

    if price_lower_low and cvd_higher_low:
      score += 2
      signals.append('Phân kỳ tăng CVD (Bullish Absorption) (+2 điểm)')

  # 3. Đánh giá ngưỡng kích hoạt AI
  if score >= 2:
    return {
        'status': 'TRIGGER_AI',
        'score': score,
        'signals': signals,
        'reason': (
            'Đạt ngưỡng định lượng. Đủ điều kiện đóng gói State Vector gửi'
            ' DeepSeek.'
        ),
    }
  else:
    return {
        'status': 'HOLD',
        'score': score,
        'signals': signals,
        'reason': (
            'Tín hiệu nhiễu hoặc điểm số chưa đạt yêu cầu. Bỏ qua gọi AI để'
            ' tiết kiệm chi phí token.'
        ),
    }
```

Cách thức hoạt động phối hợp của 3 lớp:

1. **Lớp 1 (DuckDB):** Nhận hàng triệu bản ghi trade thô, tự động gom nhóm vào RAM, tạo ra bảng `Footprint` kèm theo chỉ số `cvd` tích lũy cho từng nến.
2. **Lớp 2 (Two-Pointer TPO):** Lấy dữ liệu khối lượng từ Lớp 1 quét tìm ra ngay lập tức các vùng **POC (Điểm kiểm soát), VAH (Kháng cự giá trị) và VAL (Hỗ trợ giá trị)** mà không cần vòng lặp phức tạp.
3. **Lớp 3 (Pre-Scoring):** Kiểm tra xem khối lượng tương đối (RVOL) có đủ điều kiện không và phát hiện xem có sự **Phân kỳ CVD** hay không. Nếu đạt điểm số ≥ 2, hệ thống mới cấp phép gọi API DeepSeek V4.1 Flash.
